IEM UTM STUDENT SECTION

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

IEM Connect

Sprint 3 Presentation

Presented by: Group GGEZ

Date: 16 December 2025

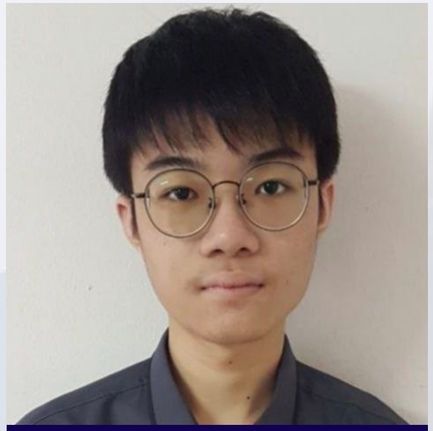
Play

Our Team

Chuah Chun Yi

Scrum Master

Lead Coordination & Stakeholder Communication



Chong Jun Hong

Developer 1

Backend Development

Muhammad Ariff bin Baharuddin

Developer 2

Frontend Development

Huzsa Syahyzal Haiqal bin Hussain

Developer 3

Frontend Development

Hrishi Vengalsetti

Developer 4

Backend Development

Sprint Overview

Sprint Goal

- To deliver Analytics Dashboard, Event Report Generation features and AI Chatbot functionality.

Sprint Duration

- 2 Weeks (Analysis + Design + Development + Testing)

MVP Objective

- Build analytics and dashboard features for admins to monitor statistics.
- Deliver event report generation to enable admins to export attendance and participation data.
- Implement AI Chatbot Interaction for members

Sprint Backlog Summary

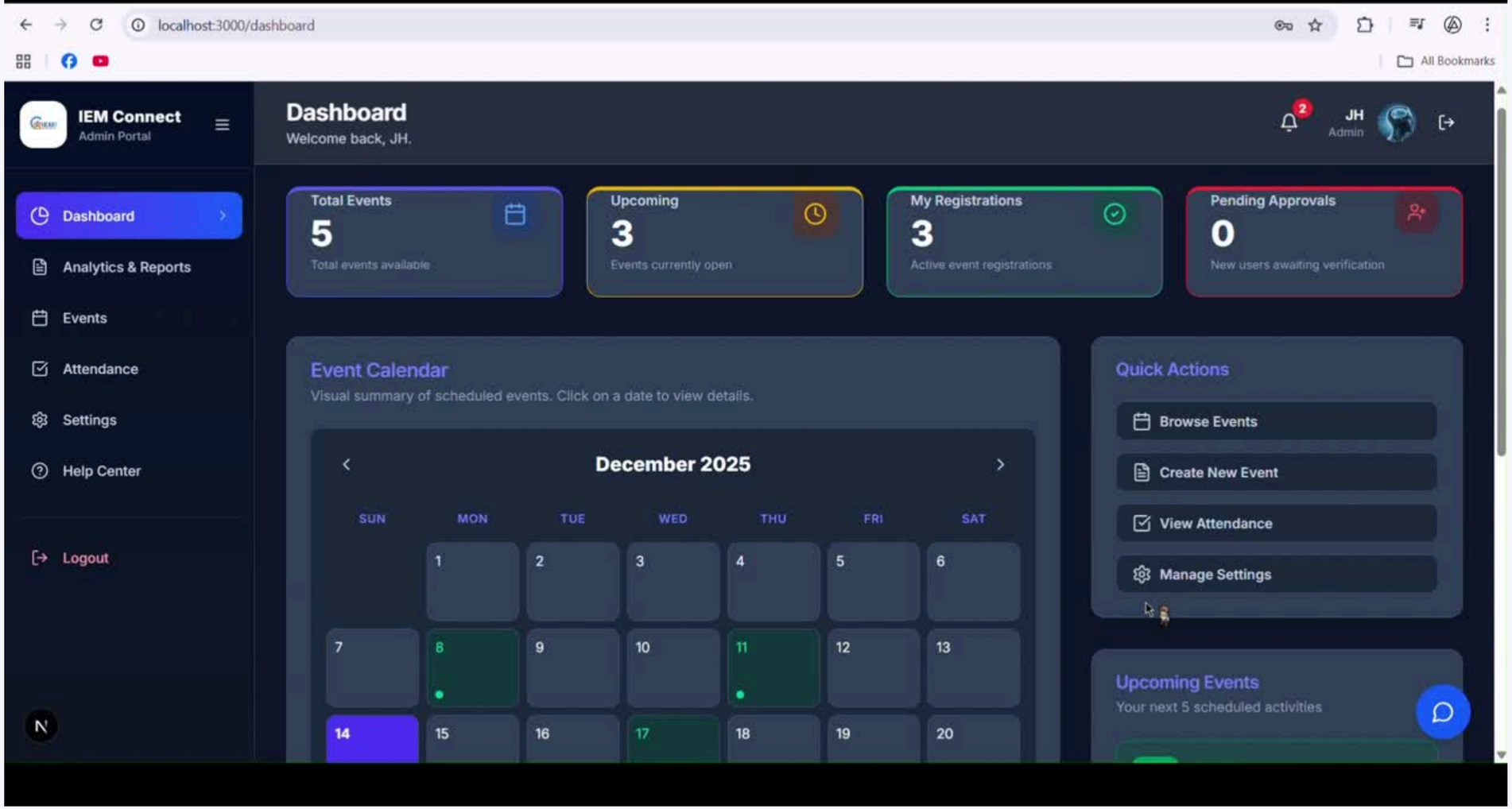
Main User Stories Implemented:

1. As an admin, I want to view users and event statistics so that I can monitor and analyze users flow and events status.
2. As an admin, I want to export attendance and participation reports so that I can easily prepare post-event documentation.
3. **As a member, I want to chat with the AI chatbot so that i can inquire information about IEM and its events. (Deferred)**

Deliverables:

- Analytics & Dashboard Module
- Event Report Generation
- **AI Chatbot Interaction (Deferred)**

MVP Demonstration



Client Feedback

Key Comments Received:

1. Overall satisfied with Analytics Dashboard and Event Report Generation.
2. UI style is acceptable but requires better consistency.
3. Calendar should support clickable dates with event pop-up view.
4. Quick action features improve usability.
5. Prefer system-wide date format dd/mm/yyyy.

Action Items for Next Sprint:

- Solve technical issues faced in implementation of AI Chatbot.
- Improve system UI and harmonize styling across all pages.
- Enhance calendar functionality with interactive event pop-ups.

Code Hightlights

Core Components

backend/controllers/reportController.js:

- Handle retrieval of overall user and event statistics
- Provide trend analysis feature
- Aggregate data for dashboard summaries

frontend/app/reports/page.tsx

- Handle retrieval of individual event statistics
- Display data for event status in chart layout
- Generate and export event reports to excel file

Example Logic

1 Analytic Dashboard Logic

```javascript
// A. User & Faculty Insights
export const getUsersInsights = async (req, res) => {
  const startTime = Date.now();
  try {
    const data = await getCached('users-insights', async () => {
      const totalUsers = await User.count();

      // Last month cutoff: end of previous month
      const now = new Date();
      const endOfLastMonth = new Date(now.getFullYear(), now.getMonth(), 0, 23, 59, 59, 999);

      const lastMonthTotalUsers = await User.count({
        where: {
          createdAt: { [Op.lte]: endOfLastMonth },
        },
      });

      const growthPercent = lastMonthTotalUsers
        ? ((totalUsers - lastMonthTotalUsers) / lastMonthTotalUsers) * 100
        : 100; // If last month had 0, treat as 100% growth

      // Faculty distribution - optimized with aggregation
      const facultyDistribution = await User.findAll({
        attributes: [
          [sequelize.fn('COALESCE', sequelize.col('faculty'), 'Unknown'), 'faculty'],
          [sequelize.fn('COUNT', sequelize.col('id')), 'count'],
        ],
        group: ['faculty'],
        raw: true,
      });

      const facultyDist = facultyDistribution.reduce((acc, row) => {
        acc[row.faculty] = parseInt(row.count);
        return acc;
      }, {});

      // Verification queue
      const pendingApprovals = await User.count({ where: { is_verified: 0 } });

      return {
        total_users: totalUsers,
        last_month_total_users: lastMonthTotalUsers,
        growth_percent: Number(growthPercent.toFixed(2)),
        faculty_distribution: facultyDist,
        pending_approvals: pendingApprovals,
      };
    }, 300); // Cache for 5 minutes

    const duration = Date.now() - startTime;
    console.log(`[PERF] getUsersInsights took ${duration}ms`);
    if (duration > 1000) {
      console.warn(`[SLOW QUERY] getUsersInsights exceeded 1s: ${duration}ms`);
    }

    res.json(data);
  } catch (error) {
    console.error("Users insights error:", error);
    res.status(500).json({ error: "Failed to compute user insights" });
  }
};
```

```javascript
// C. Attendance & Engagement
export const getAttendanceEngagement = async (req, res) => {
  const startTime = Date.now();
  try {
    const data = await getCached('attendance-engagement', async () => {
      // Parallel execution of independent queries
      const [totalAttendees, totalRegistrations, methodStats, topPerformersRaw] = await Promise.all([
        // Total attendees
        Attendance.count(),

        // Total registrations
        EventRegistration.count({
          where: { status: { [Op.in]: ["registered", "attended"] } },
        }),

        // Method split - optimized with GROUP BY
        Attendance.findAll({
          attributes: [
            'method',
            [sequelize.fn('COUNT', sequelize.col('id')), 'count'],
          ],
          group: ['method'],
          raw: true,
        }),

        // Top performers - optimized with aggregation
        Attendance.findAll({
          attributes: [
            [sequelize.fn('COUNT', sequelize.col('Attendance.id')), 'attendance_count'],
            [sequelize.col('registration.user.id'), 'user_id'],
            [sequelize.col('registration.user.name'), 'name'],
            [sequelize.col('registration.user.email'), 'email'],
            [sequelize.col('registration.user.matric_number'), 'matric_number'],
            [sequelize.col('registration.user.membership_number'), 'membership_number'],
            [sequelize.col('registration.user.faculty'), 'faculty'],
          ],
          include: [
            {
              model: EventRegistration,
              as: "registration",
              attributes: [],
              include: [
                {
                  model: User,
                  as: "user",
                  attributes: [],
                },
              ],
            },
          ],
          group: [
            sequelize.col('registration.user.id'),
            sequelize.col('registration.user.name'),
            sequelize.col('registration.user.email'),
            sequelize.col('registration.user.matric_number'),
            sequelize.col('registration.user.membership_number'),
            sequelize.col('registration.user.faculty'),
          ],
          order: [[sequelize.literal('attendance_count'), 'DESC']],
          limit: 5,
          raw: true,
        }),
      ]);
```

```javascript
      const participationRate = totalRegistrations
        ? (totalAttendees / totalRegistrations) * 100
        : 0;

      // Process method split
      const methodCounts = {
        QR: 0,
        Code: 0,
        Manual: 0,
      };
      methodStats.forEach(row => {
        if (methodCounts.hasOwnProperty(row.method)) {
          methodCounts[row.method] = parseInt(row.count);
        }
      });
      const totalMethods = methodCounts.QR + methodCounts.Code + methodCounts.Manual;
      const methodSplit = {
        QR: totalMethods ? Number(((methodCounts.QR / totalMethods) * 100).toFixed(2)) : 0,
        Code: totalMethods ? Number(((methodCounts.Code / totalMethods) * 100).toFixed(2)) : 0,
        Manual: totalMethods ? Number(((methodCounts.Manual / totalMethods) * 100).toFixed(2)) : 0,
      };

      // Process top performers - handle potential null values
      const topPerformers = topPerformersRaw
        .filter(row => row.user_id) // Filter out null users
        .map((row) => ({
          user: {
            id: row.user_id,
            name: row.name || 'Unknown',
            email: row.email || '',
            matric_number: row.matric_number || '',
            membership_number: row.membership_number || '',
            faculty: row.faculty || 'Unknown',
          },
          attendance_count: parseInt(row.attendance_count) || 0,
        }));

      return {
        total_attendees: totalAttendees,
        total_registrations: totalRegistrations,
        participation_rate: Number(participationRate.toFixed(2)),
        method_split: methodSplit,
        top_performers: topPerformers,
      };
    }, 300); // Cache for 5 minutes

    const duration = Date.now() - startTime;
    console.log(`[PERF] getAttendanceEngagement took ${duration}ms`);
    if (duration > 1000) {
      console.warn(`[SLOW QUERY] getAttendanceEngagement exceeded 1s: ${duration}ms`);
    }

    res.json(data);
  } catch (error) {
    console.error("Attendance engagement error:", error);
    res.status(500).json({ error: "Failed to compute attendance engagement" });
  }
};
```

- Aggregates user and attendance data for admin insights
- Calculates user growth, participation rate, and engagement metrics

Example Logic

2 Event Report Generation Logic

```
// Download Excel Logic (Unchanged)
const downloadExcel = (type: "overall" | "event") => {
  setDownloading(true);
  try {
    const workbook = XLSX.utils.book_new();

    if (type === "overall") {
      if (!usersInsights || !eventOps || !engagement) return;

      // Summary Sheet
      const summaryData = [
        ["Metric", "Value"],
        ["Total Users", usersInsights.total_users],
        ["User Growth % MoM", usersInsights.growth_percent],
        ["Pending Approvals", usersInsights.pending_approvals],
        ["Total Events Held", eventOps.total_events_held],
        ["Avg Attendance Rate %", engagement.participation_rate],
      ];
      const summarySheet = XLSX.utils.aoa_to_sheet(summaryData);
      XLSX.utils.book_append_sheet(workbook, summarySheet, "Summary");

      // Faculty Distribution Sheet
      const facultySheetData = [
        ["Faculty", "Attendance Count"],
        ...facultyData.map((f) => [f.name, f.value]),
      ];
      const facultySheet = XLSX.utils.aoa_to_sheet(facultySheetData);
      XLSX.utils.book_append_sheet(
        workbook,
        facultySheet,
        "Faculty Distribution"
      );

      // Trends Sheet
      const trendsSheetData = [
        ["Month", "Registrations", "Attendees"],
        ...trendData.map((t) => [t.month, t.registrations, t.attendees]),
      ];
      const trendsSheet = XLSX.utils.aoa_to_sheet(trendsSheetData);
      XLSX.utils.book_append_sheet(workbook, trendsSheet, "Trends");

      // Top Events Sheet
      const topEventsSheetData = [
        ["Event", "Date", "Attendance"],
        ...topEvents.map((e) => [
          e.event.title,
          e.event.start_date || "",
          e.attendance_count,
        ]),
      ];
```

```
      const topEventsSheet = XLSX.utils.aoa_to_sheet(topEventsSheetData);
      XLSX.utils.book_append_sheet(workbook, topEventsSheet, "Top Events");

      // Recent Activity Sheet
      const activitySheetData = [
        ["Type", "User", "Event", "Method", "Time"],
        ...recentActivity.map((a) => [
          a.type.replace("_", " "),
          a.user?.name || "",
          a.event?.title || "-",
          a.method || "-",
          new Date(a.timestamp).toLocaleString(),
        ]),
      ];
      const activitySheet = XLSX.utils.aoa_to_sheet(activitySheetData);
      XLSX.utils.book_append_sheet(
        workbook,
        activitySheet,
        "Recent Activity"
      );

      const fileName = `analytics-overall-${new Date()
        .toISOString()
        .slice(0, 10)}.xlsx`;
      XLSX.writeFile(workbook, fileName);
    } else {
      // Individual Event Stats
      if (!eventStats) return;

      const eventName = eventStats.event.title
        .replace(/[^\w\s]/gi, "")
        .replace(/\s+/g, "-");

      // Event Summary Sheet
      const summaryData = [
        ["Event Information", ""],
        ["Title", eventStats.event.title],
        ["Start Date", eventStats.event.start_date],
        ["End Date", eventStats.event.end_date],
        ["Status", eventStats.event.status],
        ["", ""],
        ["Statistics", ""],
        ["Total Registrations", eventStats.registrations.total],
        ["Total Attended", eventStats.registrations.attended],
        ["Attendance Rate %", eventStats.registrations.attendance_rate],
        ["", ""],
        ["Attendance Methods", ""],
        ["QR Code", eventStats.attendance_methods.QR],
        ["Code", eventStats.attendance_methods.Code],
        ["Manual", eventStats.attendance_methods.Manual],
      ];
      const summarySheet = XLSX.utils.aoa_to_sheet(summaryData);
      XLSX.utils.book_append_sheet(workbook, summarySheet, "Summary");
```

```
      // Participants Sheet
      const participantsSheetData = [
        [
          "Name",
          "Email",
          "Matric Number",
          "Membership Number",
          "Faculty",
          "Status",
          "Registration Date",
          "Attendance Date",
          "Method",
        ],
        ...eventStats.participants.map((p: any) => [
          p.user.name,
          p.user.email,
          p.user.matric_number || "",
          p.user.membership_number || "",
          p.user.faculty || "",
          p.status,
          p.registration_date
            ? new Date(p.registration_date).toLocaleString()
            : "",
          p.attendance_date
            ? new Date(p.attendance_date).toLocaleString()
            : "",
          p.method || "",
        ]),
      ];
      const participantsSheet =
        XLSX.utils.aoa_to_sheet(participantsSheetData);
      XLSX.utils.book_append_sheet(
        workbook,
        participantsSheet,
        "Participants"
      );

      // Faculty Distribution Sheet
      const facultySheetData = [
        ["Faculty", "Attendance Count"],
        ...Object.entries(eventStats.faculty_distribution).map(
          ([faculty, count]) => [faculty, count]
        ),
      ];
      const facultySheet = XLSX.utils.aoa_to_sheet(facultySheetData);
      XLSX.utils.book_append_sheet(
        workbook,
        facultySheet,
        "Faculty Distribution"
      );
```

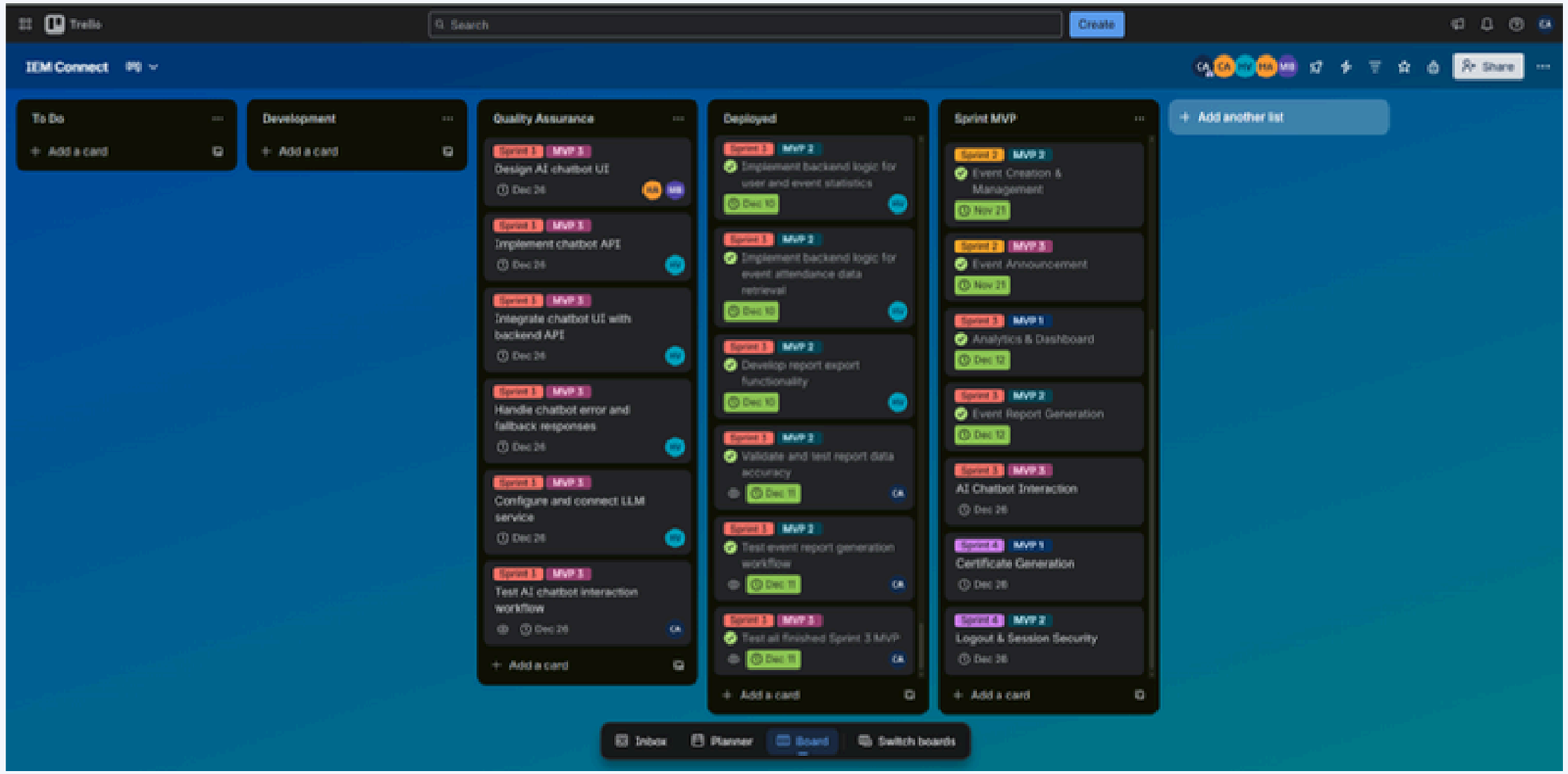
```
      // Timeline Sheet
      if (eventStats.timeline && eventStats.timeline.length > 0) {
        const timelineSheetData = [
          ["Date", "Cumulative Registrations", "Cumulative Attendance"],
          ...eventStats.timeline.map((t: any) => [
            t.date,
            t.registrations,
            t.attendance,
          ]),
        ];
        const timelineSheet =
          XLSX.utils.aoa_to_sheet(timelineSheetData);
        XLSX.utils.book_append_sheet(
          workbook,
          timelineSheet,
          "Timeline"
        );
      }

      const fileName = `analytics-event-${eventName}-${new Date()
        .toISOString()
        .slice(0, 10)}.xlsx`;
      XLSX.writeFile(workbook, fileName);
    }
  } catch (err) {
    console.error("Failed to export Excel:", err);
    alert("Failed to export Excel file. Please try again.");
  } finally {
    setDownloading(false);
  }
};
```

- Retrieves attendance records for a selected event
- Supports export to excel file for post-event documentation

Project Performance Metrics

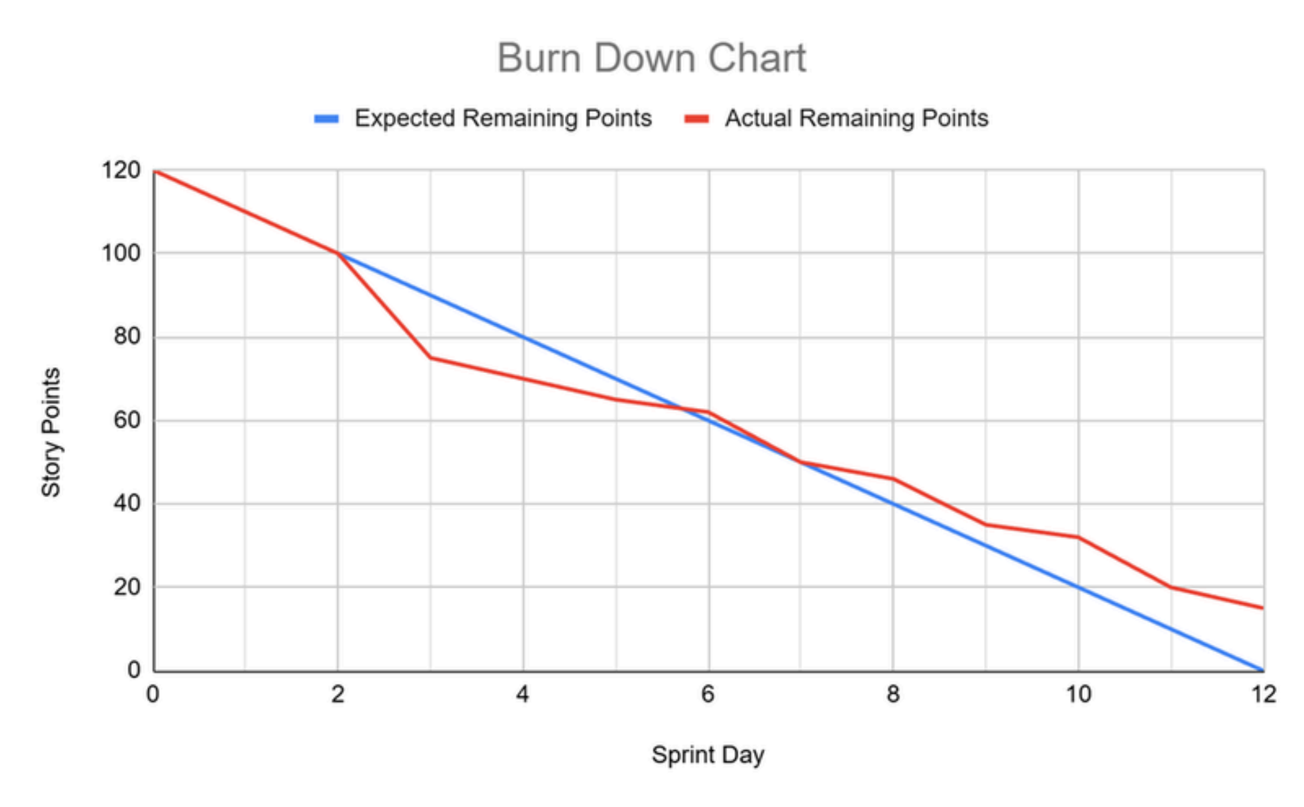
Kanban Board



Sprint 3
Total tasks: 31
Completed: 25
In progress: 6

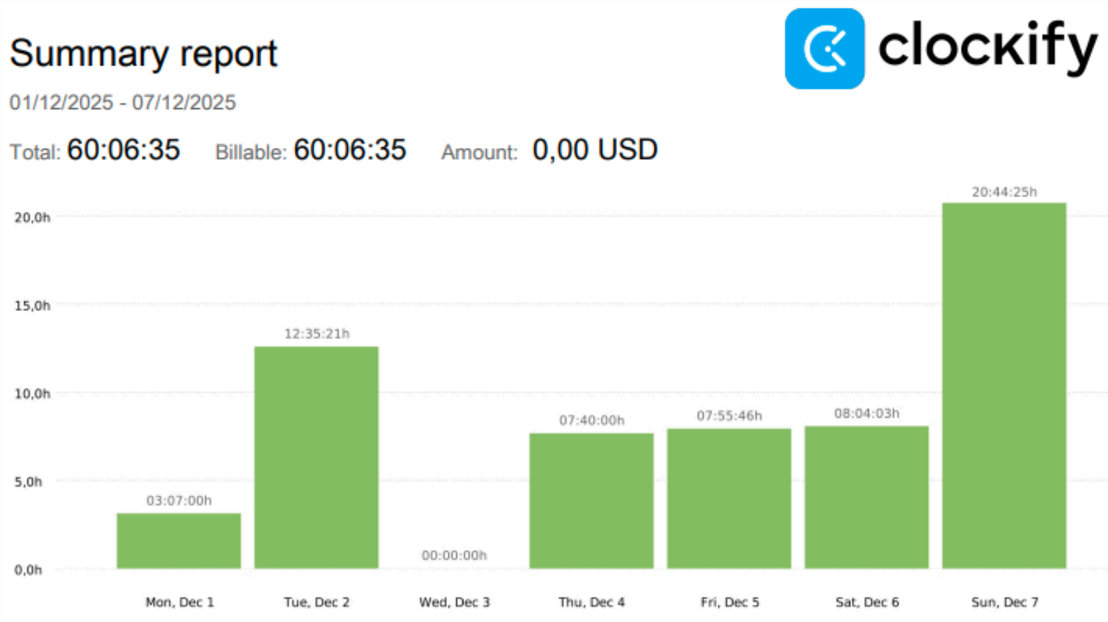
Project
Total tasks: 11
Completed: 8
In progress: 3

Burndown Chart



- Actual progress slightly above ideal line
- Remaining points due to AI Chatbot not delivered in this sprint

Clockify Summary



Total hours logged: 128.165 hours
Average per member: 25.633 hours

Next Sprint Focus

Sprint 4 Objectives:

- Resume AI Chatbot implementation
- Implement Certificate Generation
- Implement Logout & Session Security

Target Outcome:

- Enhance system security
- Provide automated certificate management
- Improve overall user experience

Conclusion

Summary:

- Sprint 3 successfully delivered Analytics Dashboard and Event Report Generation features. However, AI Chatbot integration was deferred to ensure technical readiness and better implementation quality.

Next Steps:

- Apply UI consistency improvements
- Resolve challenges on AI Chatbot implementation
- Implement Sprint 4 MVP features
- Prepare system testing

Thank You

Contact Us

Chuah Chun Yi

Scrum Master of Group GGEZ

+60 1157678402

iemconnect1@gmail.com